

LAB # 4:**BREADTH FIRST SEARCH FOR GRAPH AND TREE TRAVERSAL****Objectives:**

- To implement breadth first search (BFS) algorithm for graph and tree traversals using python

Hardware/Software Required:

Hardware: Desktop/ Notebook Computer

Software Tool: Python 2.7/ 3.6.2

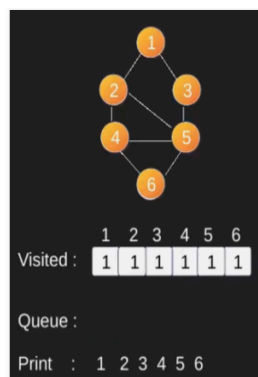
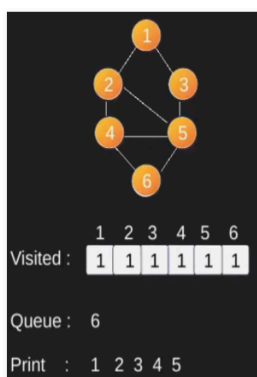
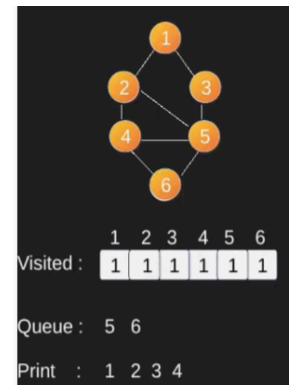
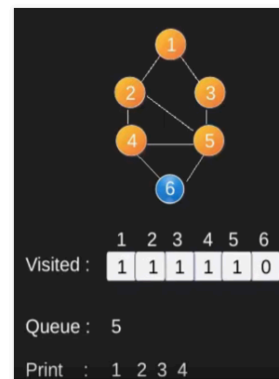
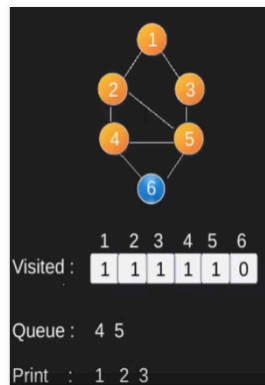
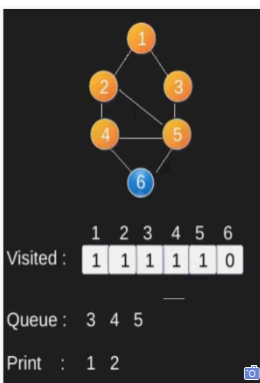
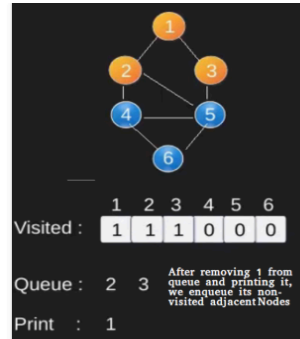
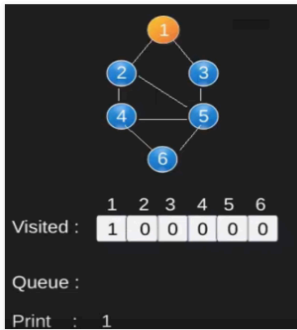
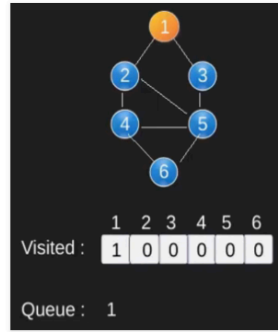
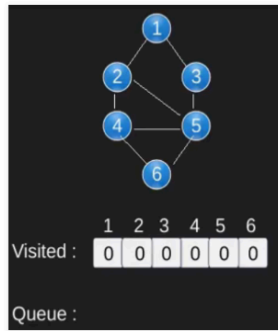
Introduction:

Breadth first search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root or some arbitrary node of a graph and explores the neighbor nodes first, before moving to the next level neighbors.

In this lab, we will implement BFS for both tree and graph traversals. Since BFS searches the sibling or neighboring nodes first before moving to the next child nodes so we will employ First In First Out (FIFO) structure in our implementation where sibling nodes will be added first before child nodes and since they are added first so they will be searched earlier as well.

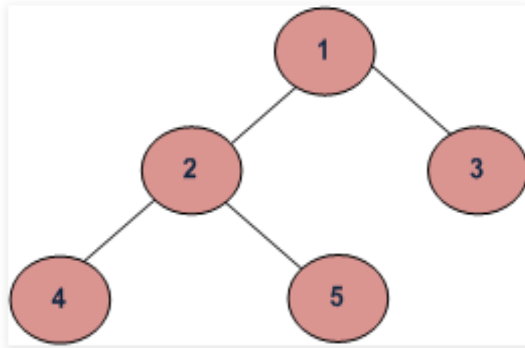
- Start from a given or random node (call it **Current/Working** node)
- Write it in a **queue**(FIFO)
Note: While writing in a queue (in case of graphs without cost), you have to write the elements in ascending order or in alphabetical order
- Check its neighboring nodes and write them in a Queue while writing the Current Node in **output**
- Update the Current Node, which is now the **First element** of Queue
- Traverse till end

Example of BFS applied on graph



Example of BFS on tree:

- In a tree, BFS is implemented Level wise and we start writing in a queue from left node



Breadth First Traversal : 1 2 3 4 5

Code for Tree:

Option 1:

```
graph={
  1:[2,3],
  2:[1,4,5],
  3:[1],
  4:[2],
  5:[2]
}
```

Option 2:

```
class Tree():
```

```
    def __init__(self):
```

```
        self.left = None
```

```
        self.right = None
```

```
        self.data = None
```

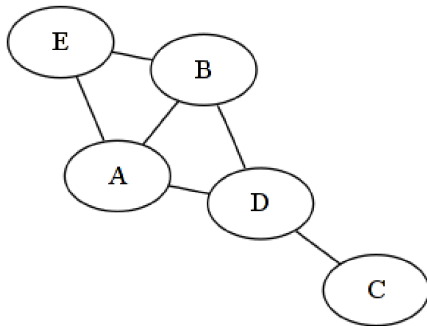
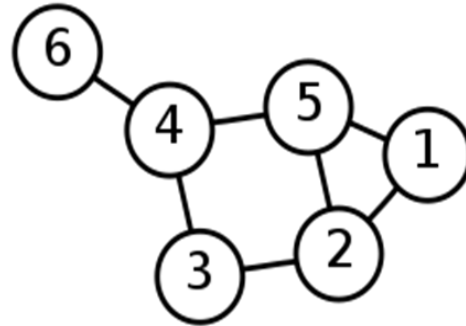
```
root = Tree()
```

```
root.data = "root"
```

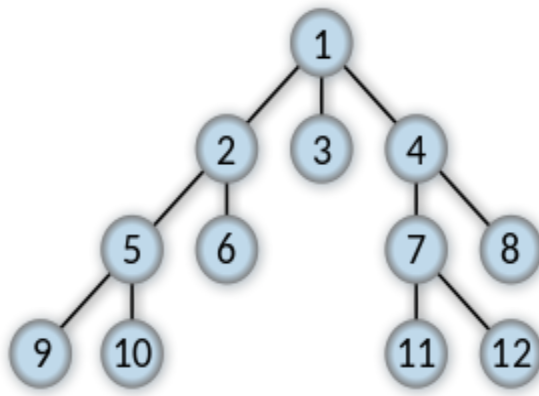
```
root.left = Tree()
root.left.data = "left"
root.right = Tree()
root.right.data = "right"
```

Lab Tasks:

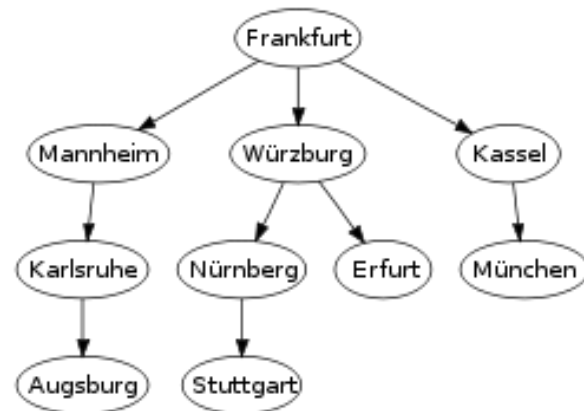
1. Implement a FIFO data structure in python.
2. Use the implemented FIFO data structure to implement BFS for Graph 1 and Graph 2 in python. The starting nodes for Graph 1 and Graph2 are '6' and 'C'.
3. Implement BFS for Tree 1 and 2 in python. The starting node is '1' for Tree 1 while the starting node is 'Frankfurt' for Tree 2.

**Graph 1****Graph 2**

Note: for creating Trees you can use the code given above



Tree 1



Tree 2

Conclusion:

Write the conclusion about this lab

NOTE: A lab journal is expected to be submitted for this lab.